- View components can use DI, access the `HttpContext`, and render partial views. The partial views should be stored in the Pages/Shared/Components/ <Name>/ folder, where Name is the name of the view component. If not specified, view components will look for a default view named Default.cshtml.
- You can create a custom `DataAnnotations` attribute by deriving from `ValidationAttribute` and overriding the `IsValid` method. You can use this to decorate your binding model properties and perform arbitrary validation.
- You can't use constructor DI with custom validation attributes. If the validation attribute needs access to services from the DI container, you must use the Service Locator pattern to load them from the validation context, using the `GetService<T>` method.
- FluentValidation is an alternative validation system that can replace the default `DataAnnotations` validation system. It is not based on attributes, which makes it easier to write custom validations for your validation rules and makes those rules easier to test.
- To create a validator for a model, create a class derived from `AbstractValidator<>` and call `RuleFor<>()` in the constructor to add validation rules. You can chain multiple requirements on `RuleFor<>()` in the same way that you could add multiple `DataAnnotations` attributes to a model.